

BÀI THỰC HÀNH

LAB 15

Đo tốc độ động cơ

Đo RPM bằng encoder hai kênh với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	Encoder trong hệ thống đo chuyển động	2
2.2	Encoder tăng dần hai kênh A/B	2
2.3	Giải mã quadrature x4 và CPR	3
2.4	Tính tốc độ RPM	3
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Kết nối điều khiển động cơ	3
3.3	Kết nối động cơ và encoder	4
3.4	Sơ đồ kết nối	4
4	EduFramework API	5
4.1	Encoder_Init()	5
4.2	Encoder_Reset()	5
4.3	Encoder_Update()	5
4.4	Encoder_GetRpm()	6
5	Bài thực hành	6
5.1	Yêu cầu	6
5.2	Chương trình	6
5.3	Giải thích chương trình	7
5.4	Kiểm chứng	8
6	Mở rộng	8
6.1	Điều chỉnh mức PWM qua UART	8
7	Tài liệu tham khảo	9

1 Giới thiệu

1.1 Tổng quan bài lab

Lab 14 đã sử dụng module điều khiển động cơ để thiết lập mức PWM, chiều quay và trạng thái dừng của động cơ DC. Tuy nhiên, giá trị PWM chỉ thể hiện lệnh điều khiển từ vi điều khiển và không cho biết trực tiếp tốc độ cơ học thực tế của động cơ.

Trong các hệ thống cần quan sát chuyển động, encoder có thể được sử dụng để tạo tín hiệu phản hồi từ trục quay. Bằng cách đếm các chuyển trạng thái của encoder trong một khoảng thời gian xác định, chương trình có thể tính tốc độ quay theo đơn vị vòng trên phút (RPM). Encoder hai kênh còn cho phép xác định chiều quay dựa trên quan hệ pha giữa hai tín hiệu [3] [5].

Bài lab này bổ sung encoder hai kênh A/B cho hệ thống động cơ của Lab 14. Động cơ được điều khiển quay liên tục ở một mức PWM cố định, trong khi EduFramework đo tín hiệu encoder và gửi tốc độ RPM thực tế tới Serial Monitor thông qua `Serial1`.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả vai trò của encoder trong đo chuyển động quay của động cơ.
2. Giải thích nguyên lý cơ bản của encoder tăng dần hai kênh A/B và giải mã quadrature x4.
3. Xác định số count trên một vòng quay từ thông số xung của encoder.
4. Sử dụng Encoder Device API để cập nhật và đọc tốc độ RPM của động cơ.
5. Hiển thị tốc độ động cơ theo thời gian thực trên Serial Monitor.

2 Kiến thức nền

2.1 Encoder trong hệ thống đo chuyển động

Encoder là thiết bị tạo tín hiệu điện tương ứng với chuyển động cơ học. Trong các hệ thống quay, encoder thường được gắn với trục động cơ để cung cấp thông tin về vị trí, số bước chuyển động, chiều quay hoặc tốc độ [1] [5].

Encoder có thể sử dụng nhiều nguyên lý cảm biến khác nhau. Một số công nghệ thường gặp gồm encoder quang, encoder từ và encoder điện dung [6]. Bài lab này sử dụng encoder quang tích hợp trực tiếp trên động cơ DC.

Trong encoder quang, một đĩa có các vùng hoặc khe tuần hoàn quay cùng trục động cơ. Nguồn sáng và phần tử thu quang biến sự thay đổi ánh sáng thành chuỗi xung điện. Cách bố trí hai kênh tín hiệu giúp hệ thống xác định không chỉ số bước chuyển động mà còn cả chiều quay [2].

2.2 Encoder tăng dần hai kênh A/B

Encoder sử dụng trong bài lab cung cấp hai kênh A và B. Hai tín hiệu có dạng tuần hoàn và lệch pha nhau. Thứ tự thay đổi giữa A và B phụ thuộc vào chiều quay, nhờ đó bộ giải mã có thể xác định chuyển động theo hai hướng [3].

Có thể hình dung các trạng thái của hai kênh dưới dạng các cặp logic:

00 → 01 → 11 → 10 → 00

Khi chiều quay đảo lại, thứ tự chuyển trạng thái cũng đảo lại. EduFramework theo dõi các chuyển trạng thái hợp lệ và tăng hoặc giảm biến đếm encoder tương ứng [8].

2.3 Giải mã quadrature x4 và CPR

Một chu kỳ đầy đủ của encoder hai kênh có bốn chuyển cạnh có thể được sử dụng để đếm. Khi cả cạnh lên và cạnh xuống của hai kênh A và B đều được sử dụng, phương pháp này được gọi là quadrature x4 [3].

Encoder tích hợp trên động cơ của bài lab tạo 30 xung trên một vòng quay. EduFramework sử dụng giải mã x4, vì vậy số count trên một vòng cơ học là:

$$30 \times 4 = 120$$

Do đó, giá trị `countsPerRevolution` được truyền cho `Encoder_Init()` là 120U.

Ghi chú

Giá trị CPR truyền vào Encoder Device API phải là số count sau giải mã trên một vòng cơ học, không chỉ là số xung danh định của encoder.

2.4 Tính tốc độ RPM

Tốc độ quay có thể được tính từ số count thay đổi trong một khoảng thời gian. Nguyên tắc chung là lấy số count mới trong cửa sổ đo, quy đổi theo CPR và thời gian đo để thu được số vòng trên phút [4].

Trong EduFramework, phép tính có thể biểu diễn theo quan hệ:

$$\text{RPM} = |\Delta\text{Count}| \times 60000 / (\text{CPR} \times \Delta t_{\text{ms}})$$

Trong đó:

- `ΔCount` là số count thay đổi trong cửa sổ đo;
- `CPR` là số count trên một vòng quay;
- `Δt_ms` là thời gian của cửa sổ đo theo millisecond;
- `60000` là số millisecond trong một phút.

RPM được trả về dưới dạng độ lớn dương. Chiều quay được quản lý riêng bởi Encoder Device API.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Lab 15 tiếp tục sử dụng hệ thống điều khiển động cơ của Lab 14 và bổ sung phần encoder tích hợp trên động cơ. Module TB6612FNG và nguồn ngoài vẫn đảm nhiệm phần điều khiển công suất cho động cơ.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
TB6612FNG Motor Driver Module	Module điều khiển động cơ đã sử dụng trong Lab 14.
DC Motor with Encoder	Động cơ DC tích hợp encoder quang hai kênh A/B, 30 xung trên một vòng.
External Motor Power Supply	Nguồn ngoài cấp cho động cơ thông qua TB6612FNG.
Jumper wires	Kết nối các tín hiệu điều khiển, encoder và nguồn.
USB Cable	Kết nối MaaZEDU với máy tính để nạp chương trình và sử dụng Serial Monitor.

Bảng 15.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Kết nối điều khiển động cơ

Phần điều khiển động cơ giữ nguyên cấu hình của Lab 14:

Thành phần	Logical Pin	MCU Pin	Chức năng
TB6612FNG PWMA	GPI02	PTD14	PWM Output
TB6612FNG AIN1	GPI03	PTD13	Digital Output
TB6612FNG AIN2	GPI04	PTD12	Digital Output
TB6612FNG STBY	GPI05	PTD11	Digital Output

Bảng 15.3.2. Các tín hiệu điều khiển động cơ được sử dụng lại từ Lab 14

Chi tiết về H-bridge, nguồn VCC, nguồn VM và điều khiển PWM đã được trình bày trong Lab 14 nên không được lặp lại trong bài lab này.

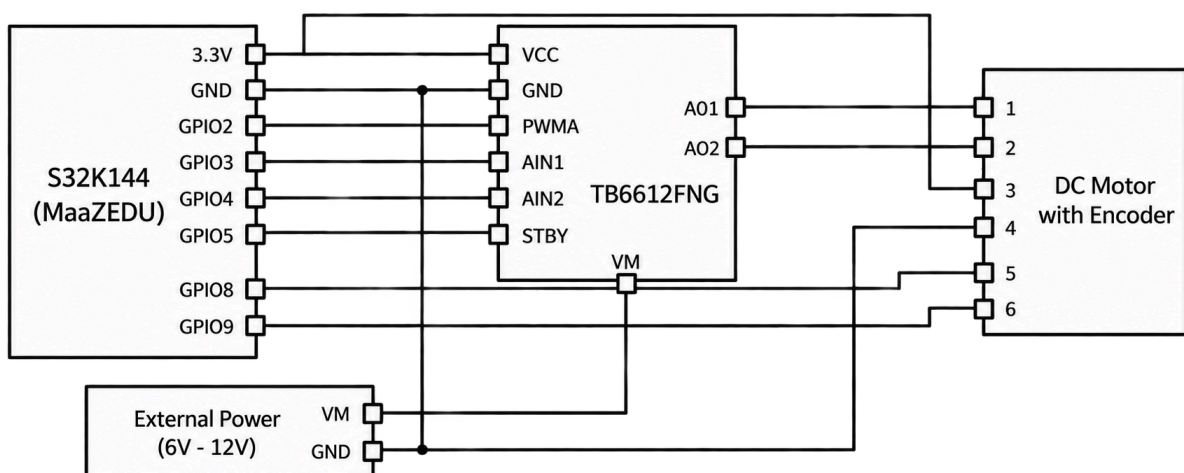
3.3 Kết nối động cơ và encoder

Động cơ sử dụng trong bài lab có đầu nối sáu chân. Theo cấu hình phần cứng đã được kiểm chứng, các chân được kết nối như sau:

Chân động cơ	Kết nối	Chức năng
Pin 1	A01	Đầu thứ nhất của động cơ.
Pin 2	A02	Đầu thứ hai của động cơ.
Pin 3	3.3V	Nguồn encoder.
Pin 4	GND	Mass encoder.
Pin 5	GPI08	Encoder Channel A.
Pin 6	GPI09	Encoder Channel B.

GPI08 và GPI09 được chọn làm hai đầu vào encoder vì các Logical Pin này hỗ trợ Digital Input và external interrupt trong cấu hình hiện tại của EduFramework [7] [8].

3.4 Sơ đồ kết nối



Hình 15.3.1. Sơ đồ kết nối động cơ DC tích hợp encoder với MaaZEDU

4 EduFramework API

Lab 15 sử dụng lại DC Motor Device API của Lab 14 để tạo chuyển động và tập trung vào Encoder Device API mới. Các API chính trong bài thực hành gồm `Encoder_Init()`, `Encoder_Reset()`, `Encoder_Update()` và `Encoder_GetRpm()` [8].

4.1 Encoder_Init()

`Encoder_Init()` khởi tạo hai kênh encoder và cấu hình số count trên một vòng quay.

Encoder_Init

Khởi tạo encoder hai kênh và cấu hình hệ số count trên một vòng cơ học.

Cú pháp

```
Encoder_Init(channelAPin, channelBPin, countsPerRevolution);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>channelAPin</code>	Logical Pin	Chân kết nối với encoder Channel A.
<code>channelBPin</code>	Logical Pin	Chân kết nối với encoder Channel B.
<code>countsPerRevolution</code>	Count / revolution	Số count sau giải mã trên một vòng cơ học.

Giá trị trả về

Trả về `1U` nếu khởi tạo thành công và `0U` nếu cấu hình không hợp lệ.

Ví dụ:

```
Encoder_Init(GPIO8, GPIO9, 120U);
```

4.2 Encoder_Reset()

`Encoder_Reset()` đưa bộ đếm tích lũy, RPM và trạng thái chiều quay về trạng thái ban đầu.

Encoder_Reset

Đặt lại trạng thái đo của encoder.

Cú pháp

```
Encoder_Reset();
```

Giá trị trả về

Không trả về giá trị.

4.3 Encoder_Update()

`Encoder_Update()` cập nhật phép đo tốc độ dựa trên count encoder và thời gian đã trôi qua. Hàm hoạt động theo kiểu không chặn và cần được gọi lặp lại trong vòng lặp chính.

Encoder_Update

Cập nhật giá trị RPM và chiều quay mới nhất của encoder.

Cú pháp

```
Encoder_Update();
```

Giá trị trả về

Không trả về giá trị.

Trong phiên bản EduFramework hiện tại, RPM được cập nhật theo một cửa sổ đo nội bộ. Application vẫn có thể tiếp tục thực hiện các tác vụ khác giữa các lần cập nhật.

4.4 Encoder_GetRpm()

`Encoder_GetRpm()` trả về tốc độ quay mới nhất đã được tính bởi `Encoder_Update()`.

Encoder_GetRpm

Đọc tốc độ quay mới nhất của encoder.

Cú pháp

```
Encoder_GetRpm();
```

Giá trị trả về

Tốc độ quay theo đơn vị vòng trên phút (RPM).

EduFramework còn cung cấp `Encoder_GetCount()` và `Encoder_GetDirection()` để đọc tổng count tích lũy và chiều quay. Hai API này không bắt buộc trong bài thực hành chính nhưng có thể được sử dụng khi cần quan sát thêm trạng thái của encoder.

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình cho động cơ quay liên tục ở mức điều khiển khoảng 50%, đo tốc độ bằng encoder và hiển thị RPM lên Serial Monitor.

Chương trình cần đáp ứng các yêu cầu sau:

- Khởi tạo Serial1 ở 9600 bps.
- Khởi tạo hệ thống điều khiển động cơ với GPIO2 đến GPIO5.
- Khởi tạo encoder với Channel A tại GPIO8, Channel B tại GPIO9 và 120 count trên một vòng.
- Đặt mức điều khiển tốc độ động cơ bằng 128.
- Cho động cơ quay theo chiều Forward liên tục.
- Gọi `Encoder_Update()` liên tục trong vòng lặp chính.
- Hiển thị RPM lên Serial Monitor sau mỗi 200 ms.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình như sau:

```
#include "Arduino.h"
#include "hardware_serial.h"
#include "dc_motor.h"
#include "encoder.h"

int main(void)
{
    uint32_t previousTime = 0U;

    setup();

    Serial1_begin(9600U);

    DCMotor_Init(GPIO2, GPIO3, GPIO4, GPIO5);

    if (0U == Encoder_Init(GPIO8, GPIO9, 120U))
    {
        Serial1_println("Encoder initialization failed.");

        while (1)
        {
        }
    }

    Encoder_Reset();

    DCMotor_SetSpeed(128U);
    DCMotor_Forward();

    Serial1_println("Motor speed measurement started.");

    while (1)
    {
        Encoder_Update();

        if ((millis() - previousTime) >= 200U)
        {
            previousTime = millis();

            Serial1_print("RPM: ");
            Serial1_printFloat(Encoder_GetRpm());
            Serial1_println("");
        }
    }

    return 0;
}
```

Mã nguồn 15.5.1. Chương trình đo tốc độ động cơ bằng encoder

5.3 Giải thích chương trình

Sau `setup()`, `Serial1_begin(9600U)` khởi tạo Serial Monitor. Phần điều khiển động cơ tiếp tục sử dụng cấu hình đã học trong Lab 14.

`Encoder_Init(GPIO8, GPIO9, 120U)` cấu hình hai kênh encoder và khai báo 120 count trên một vòng. Nếu khởi tạo không thành công, chương trình in thông báo lỗi và không cho tiếp tục thực hiện phần đo.

Sau khi `Encoder_Reset()` đưa trạng thái đo về ban đầu, `DCMotor_SetSpeed(128U)` thiết lập mức điều khiển khoảng 50% và `DCMotor_Forward()` cho động cơ quay liên tục.

Trong vòng lặp chính, `Encoder_Update()` được gọi liên tục để cập nhật tốc độ. Cứ mỗi 200 ms, chương trình đọc giá trị mới nhất bằng `Encoder_GetRpm()` và gửi kết quả tới Serial Monitor.

Việc tách thời gian cập nhật Serial khỏi phép đo encoder giúp vòng lặp chính tiếp tục gọi `Encoder_Update()` thường xuyên thay vì dừng chương trình bằng `delay()`.

5.4 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board. Cấp nguồn ngoài cho động cơ, mở Serial Monitor ở 9600 bps và quan sát kết quả.

Ví dụ:

```
Motor speed measurement started.  
RPM: 2115.000  
RPM: 2130.000  
RPM: 2125.000  
RPM: 2130.000
```

Giá trị RPM thực tế phụ thuộc vào động cơ, điện áp nguồn, tải cơ học và điều kiện vận hành nên không bắt buộc phải trùng với ví dụ.

KẾT QUẢ MONG ĐỢI

Động cơ quay liên tục theo một chiều ở mức điều khiển 128. Serial Monitor hiển thị giá trị RPM được đo từ encoder và cập nhật định kỳ khoảng 200 ms. Khi tốc độ động cơ ổn định, các giá trị RPM liên tiếp dao động quanh một mức tương đối ổn định.

6 Mở rộng

Bài thực hành chính sử dụng một mức PWM cố định để tập trung vào cách đo tốc độ bằng encoder. Khi kết hợp dữ liệu đầu vào từ UART với phản hồi RPM, chương trình có thể được mở rộng thành một ứng dụng điều khiển tốc độ tương tác.

6.1 Điều chỉnh mức PWM qua UART

Mở rộng chương trình để người dùng nhập một giá trị từ 0 đến 255 thông qua Serial Monitor. Giá trị nhận được được sử dụng làm mức điều khiển mới cho `DCMotor_SetSpeed()`.

Sau mỗi lần thay đổi giá trị điều khiển, encoder tiếp tục đo tốc độ thực tế và Serial Monitor hiển thị RPM tương ứng.

Lưuồng xử lý mong muốn:

UART Input → PWM Command → DC Motor → Encoder → Measured RPM

Gợi ý:

- Kiểm tra dữ liệu nhận được từ `Serial1`.
- Chuyển dữ liệu nhập thành giá trị số.
- Kiểm tra giá trị nằm trong miền 0..255.
- Cập nhật mức điều khiển bằng `DCMotor_SetSpeed()`.
- Tiếp tục gọi `Encoder_Update()` trong vòng lặp chính.
- Hiển thị cả giá trị PWM đang sử dụng và RPM thực tế để so sánh.

Không yêu cầu sử dụng thuật toán tự động hiệu chỉnh PWM theo RPM. Người dùng trực tiếp thay đổi lệnh điều khiển và quan sát phản hồi tốc độ của động cơ.

KẾT QUẢ MONG ĐỢI

Người dùng có thể nhập các mức điều khiển khác nhau qua Serial Monitor. Động cơ thay đổi tốc độ tương ứng và encoder cung cấp giá trị RPM thực tế để quan sát ảnh hưởng của từng mức PWM.

7 Tài liệu tham khảo

- [1] University of Michigan, *EECS 461 - Embedded Control Systems* , Department of Electrical Engineering and Computer Science.
- [2] Eindhoven University of Technology, *Incremental Optical Encoder and Position Measurement* , Technical Publication.
- [3] National Instruments, *Hardware Encoder Measurements: How-To Guide* , NI Technical Documentation.
- [4] Texas Instruments, *eQEP Position and Speed Measurement* , MCU+ SDK Documentation.
- [5] Rockwell Automation, *Encoder/Counter Modules User Manual* , Industrial Automation Documentation.
- [6] CUI Devices, *Capacitive, Magnetic, and Optical Encoders: Comparing the Technologies* , Technical Article.
- [7] MaaZEDU, *MaaZEDU Development Board Guide* , Rev. 1.0, 2025.
- [8] EduFramework, *Encoder Device API* , EduFramework Source Code.